ALBATROSS — Terminal Launch Guide

Chattogram Port SITL + Gazebo chase-cam feed

August 25, 2026

1 What launches

One command brings up the whole stack. The pieces:

- **GCS web server** (`server.py`) on `http://localhost:8080` — map, mission planner, telemetry, feed proxy.
- **PX4 SITL + Gazebo** (`gz_x500`) in the `chattogram` world, home at the port.
- **Chase camera** (`chase_cam.py`) — spawns an invisible camera that follows the drone from behind/above and publishes `/chase_cam`.
- **Camera bridge** (`cam_bridge.py`) — encodes `/chase_cam` to MJPEG on `:8091`; the web UI's feed proxies it.
- **mavlink2rest** — MAVLink → REST on `:8088` for telemetry/commands.

2 Launch from the terminal

One-shot (recommended) — from the project root (e.g. `~/albatross`):

```
bash launch/start_all.sh
```

This sources the ROS 2 / Gazebo environment, kills any stale instances, then starts the web server and `launch/launch_chattogram_clean.sh` (SITL + chase cam + bridge), all detached so they survive the terminal closing. Paths are self-locating, so the folder can live anywhere and be named anything (override with `MARITIME_ROOT`). PX4-Autopilot and mavlink2rest are expected as symlinks in `external/`.

Then open http://localhost:8080 and click the `Gazebo feed` button.

Warning: do **not** paste the launcher's contents inline. It runs `pkill -f "cam_bridge.py"` etc.; if those strings are in your shell's own command line, `pkill` kills the shell. Always run it as a *file*.

3 Manual launch (if you want the pieces separately)

```
# run from the project root
# 1) environment
source /opt/ros/humble/setup.bash
source external/PX4-Autopilot/Tools/simulation/gz/setup.sh
```

```
# 2) GCS web server (:8080)
python3 gcs/server.py 8080

# 3) SITL + chase cam + bridge + mavlink2rest (in another terminal)
bash launch/launch_chattogram_clean.sh
```

4 Verify it is up (∼15 s warmup)

```
# web server responding
curl -s http://localhost:8080/api/kml | head -c 60

# telemetry connected (expect "connected": true, 3D fix)
curl -s http://localhost:8080/api/telemetry

# camera feed producing fresh frames
curl -s http://localhost:8091/healthz     # -> {"ok": true, "fresh": true}
```

The chase-cam feed needs ∼15 s after boot (Gazebo render-engine warmup); until then the feed shows a "Waiting for Gazebo camera…" placeholder.

5 Arming & mission flow

The launcher *auto-applies* the SITL-only arming bypass plus the smooth-flight and yaw params about 12 s after boot, so no manual step is normally needed. To set them by hand, write to the pxh FIFO:

```
for p in "COM_RC_IN_MODE 4" "CBRK_SUPPLY_CHK 894281" \
         "NAV_DLL_ACT 0" "NAV_RCL_ACT 0" "COM_RCL_EXCEPT 7"; do
  echo "param set $p" > ~/.px4_stdin; sleep 0.4
done
echo "param save" > ~/.px4_stdin
```

Full flow in the UI: **Add waypoints** (each with its own height) → **Plan route** (routes around no-fly zones) → **Upload** → **Arm** → **Start**.

6 Stop everything

```
# from the UI: click "Stop SITL", or from the terminal:
pkill -f "gz sim"; pkill -f cam_bridge.py; pkill -f chase_cam.py
pkill -f mavlink2rest; pkill -f "bin/px4"
```

7 Why the feed is a chase cam (not a screen grab)

The old feed screen-scraped the desktop with `ffmpeg x11grab`, which returns an all-black frame under this **Wayland** session (XWayland is rootless). The current feed renders a real Gazebo camera on the GPU and streams it, so it works under Wayland, Xorg, or fully headless. `chase_cam.py`

moves that camera to track the drone every tick using the same `set_pose` service the project's `imu_sim_bridge` uses.

Ports at a glance

`8080`	GCS web UI + API + `/video.mjpeg` proxy
`8091`	chase-cam MJPEG bridge
`8088`	mavlink2rest REST API
`14555`	PX4 → mavlink2rest (UDP)
`14550`	PX4 → QGroundControl (UDP, zero-config)